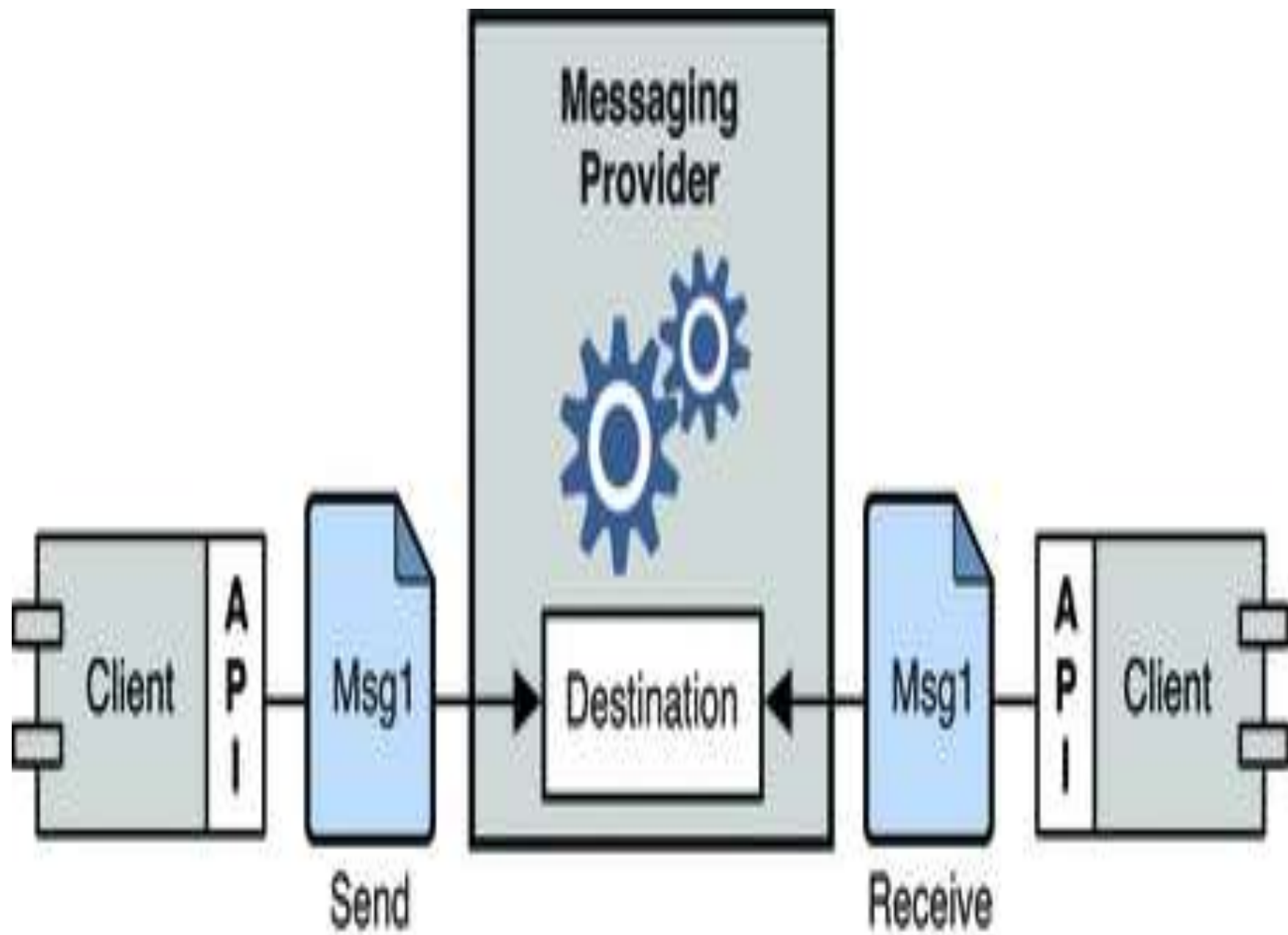


Multi Tier Computing

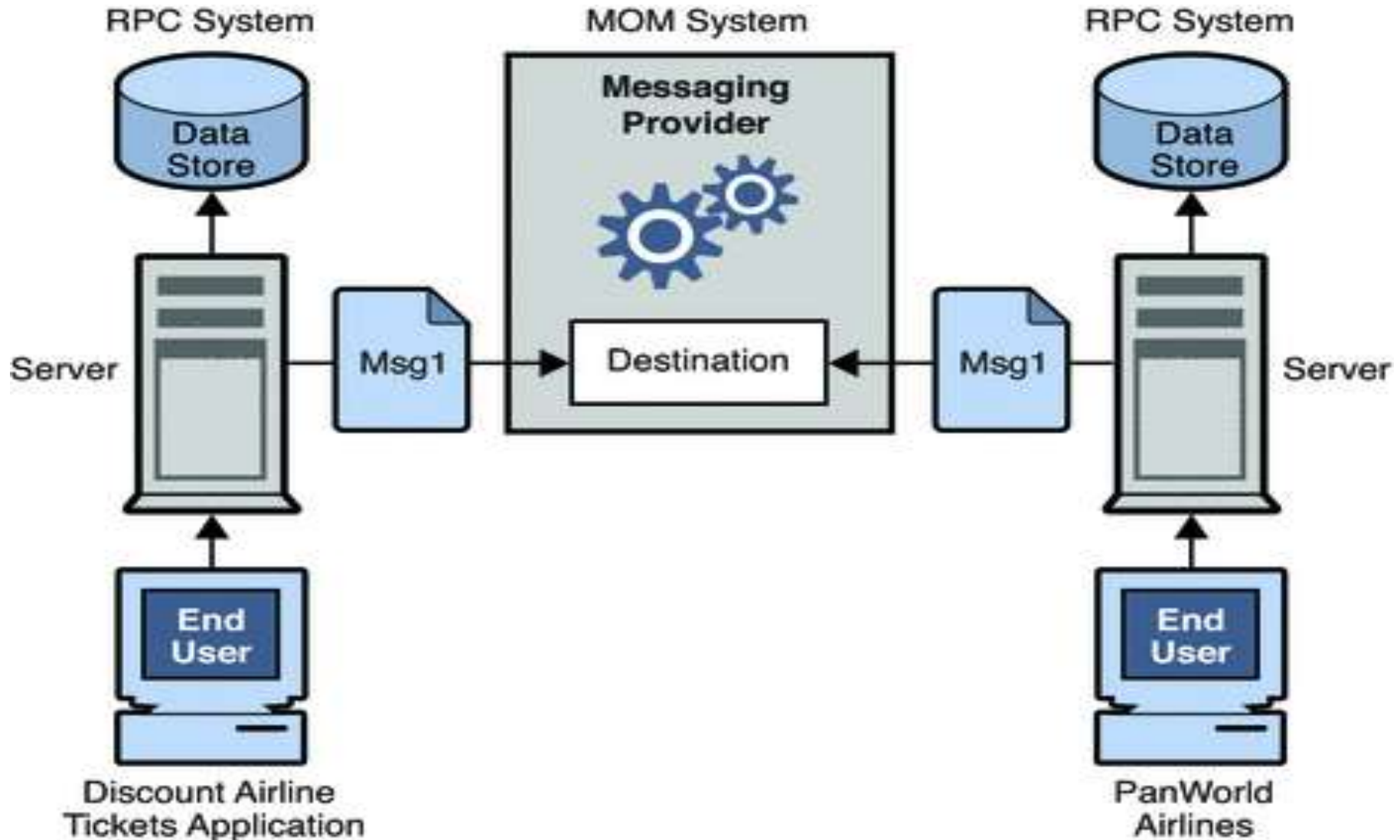
Types of Middleware

Types of Middleware

- **Message-oriented middleware**
- Message-oriented middleware (MOM) is middleware
- where transactions or event notifications are delivered between disparate systems or components by way of messages,
- often via an enterprise messaging system.
- With MOM, messages sent to the client are collected and stored until they are acted upon, while the client continues with other processing.



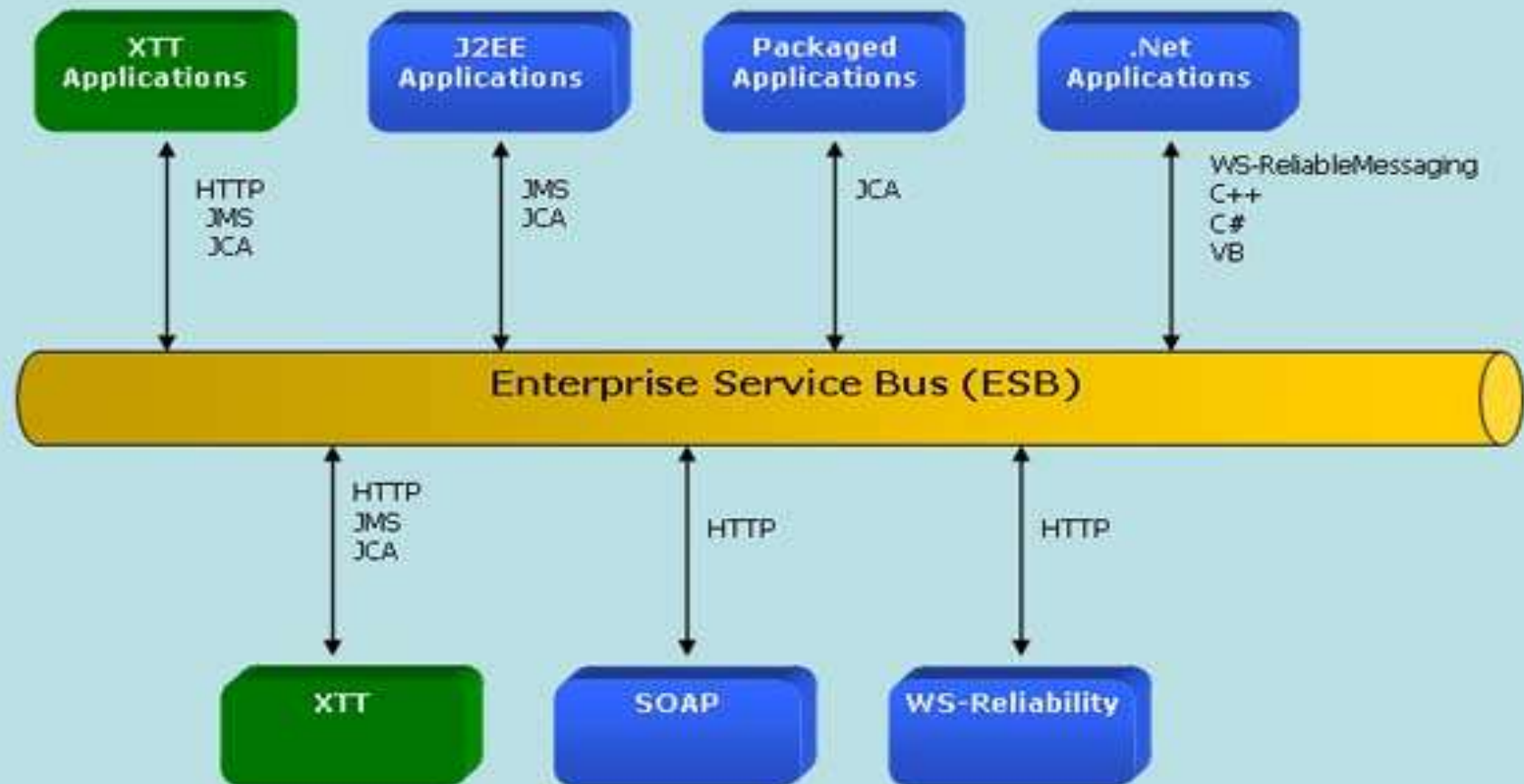
Combining RPC and MOM systems



Enterprise messaging system

- An enterprise messaging system is a type of middleware that facilitates message passing between disparate systems or components in standard formats,
- often using XML, SOAP or web services.
- As part of an enterprise messaging system, message broker software may queue, duplicate, translate and deliver messages to disparate systems or components in a messaging system.

ESB architecture



EJB- Enterprise JavaBeans
JMS- Java Messaging Service
JCA- J2EE Connector Architecture

Enterprise service bus

- Enterprise service bus (ESB) is defined by the Burton Group as "some type of integration middleware product that supports both message-oriented middleware and Web services".

Functional requirements of an ESB



Intelligent middleware

- **Intelligent Middleware (IMW)** provides real-time intelligence and event management through intelligent agents.
- The IMW manages the real-time processing of high volume sensor signals and turns these signals into intelligent and actionable business information.
- The actionable information is then delivered in end-user power dashboards to individual users or is pushed to systems within or outside the enterprise.
- It is able to support various heterogeneous types of hardware and software and provides an API for interfacing with external systems.
- It should have a highly scalable, distributed architecture which embeds intelligence throughout the network to transform raw data systematically into actionable and relevant knowledge.
- It can also be packaged with tools to view and manage operations and build advanced network applications most effectively.

Content-centric middleware

- **Content-centric middleware** offers a simple provider-consumer abstraction through which applications can issue requests for uniquely identified content, without worrying about where or how it is obtained.
- **Juno** is one example, which allows applications to generate content requests associated with high-level delivery requirements.
- The middleware then adapts the underlying delivery to access the content from sources that are best suited to matching the requirements.
- This is therefore similar to Publish/subscribe middleware, as well as the Content-centric networking paradigm.
- Example: **DDS (data distribution services)**

Remote procedure call

Remote procedure call

- Remote procedure call middleware enables a client to use services running on remote systems. The process can be synchronous or asynchronous.

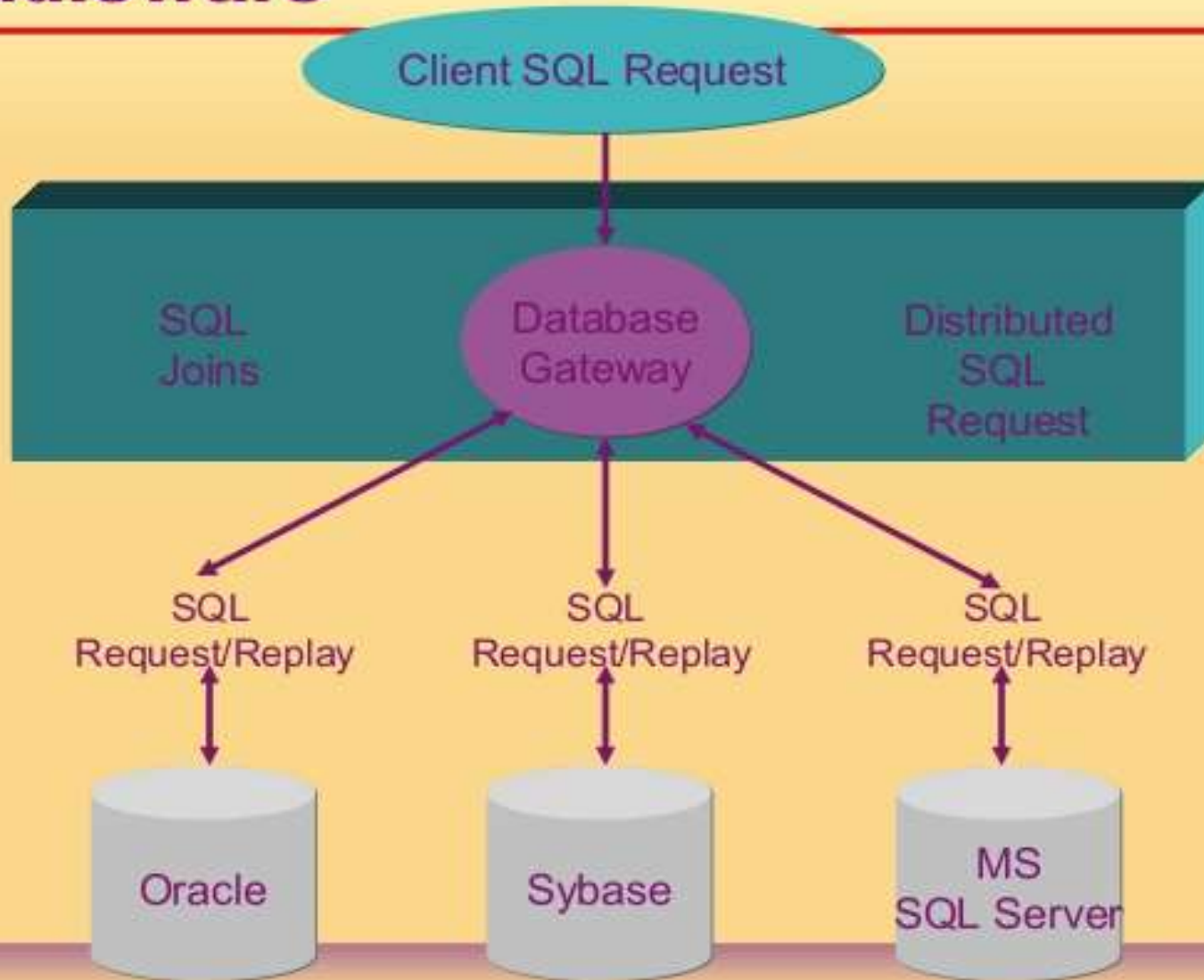
Object request broker

- With object request broker middleware, it is possible for applications to send objects and request services in an object-oriented system.

SQL-oriented data access

- SQL-oriented Data Access is middleware between applications and database servers.

Integration with Data Access Middleware

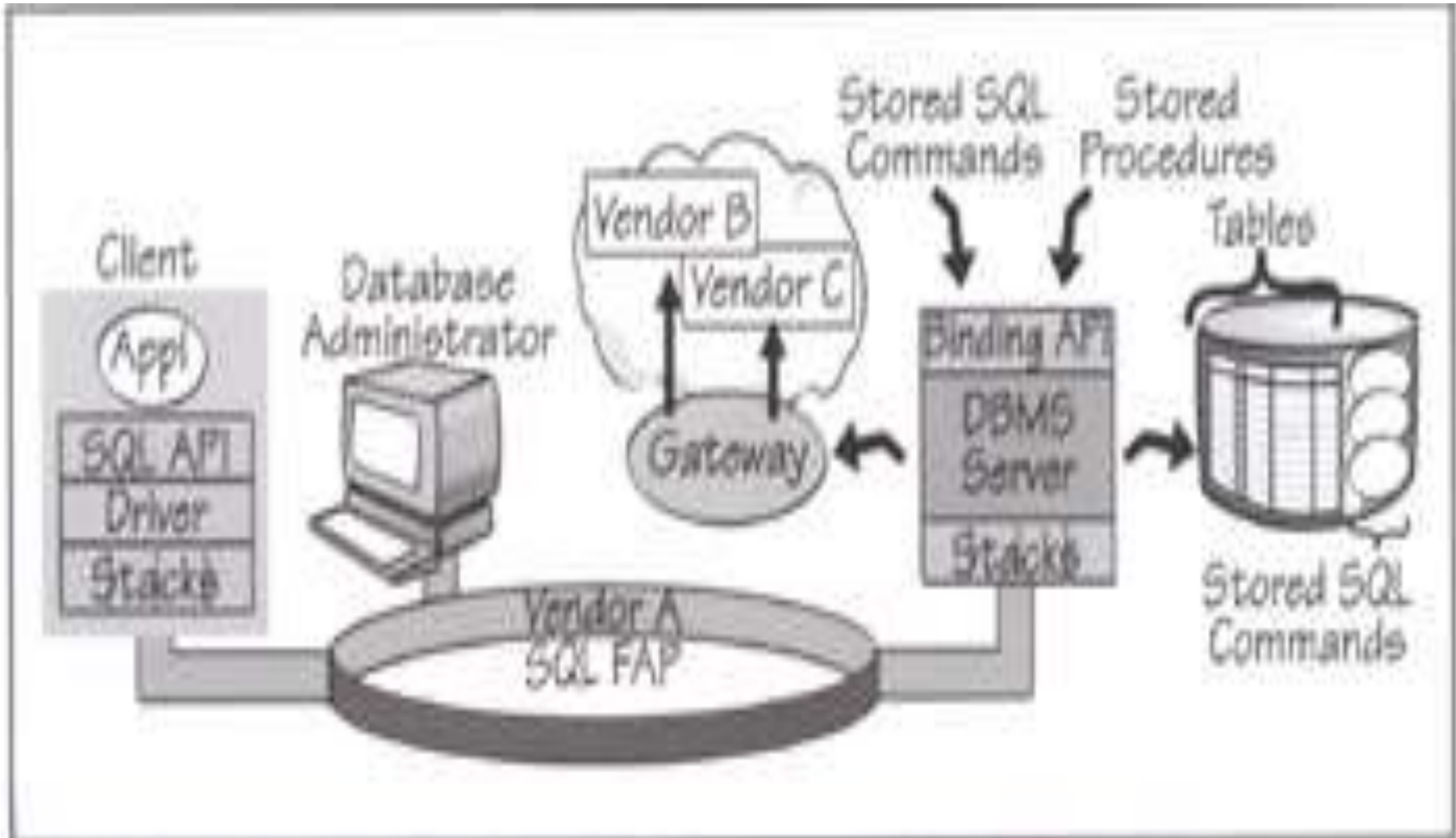


Embedded middleware

- Embedded middleware **provides communication services and software/firmware integration interface** that operates between embedded applications, the embedded operating system, and external applications.
- **Transaction processing monitors** – provides tools and an environment to develop and deploy distributed applications Programming controls most middleware facilities other than transactions
- **Application servers** – software installed on a computer to facilitate the serving (running) of other applications. Programming controls most middleware facilities

Application servers

SQL integration/interoperability: Single Vendor option



A vendor-proprietary SQL driver

- This is a thin client run-time element that accepts the API calls, formats an SQL message, and handles the exchanges with the server.
- The format of the SQL message and the handshake are known affectionately as the **FAP**, which stands for **Format and Protocols**. The SQL FAPs are typically vendor-defined.

FAP support for multiple protocol stacks.

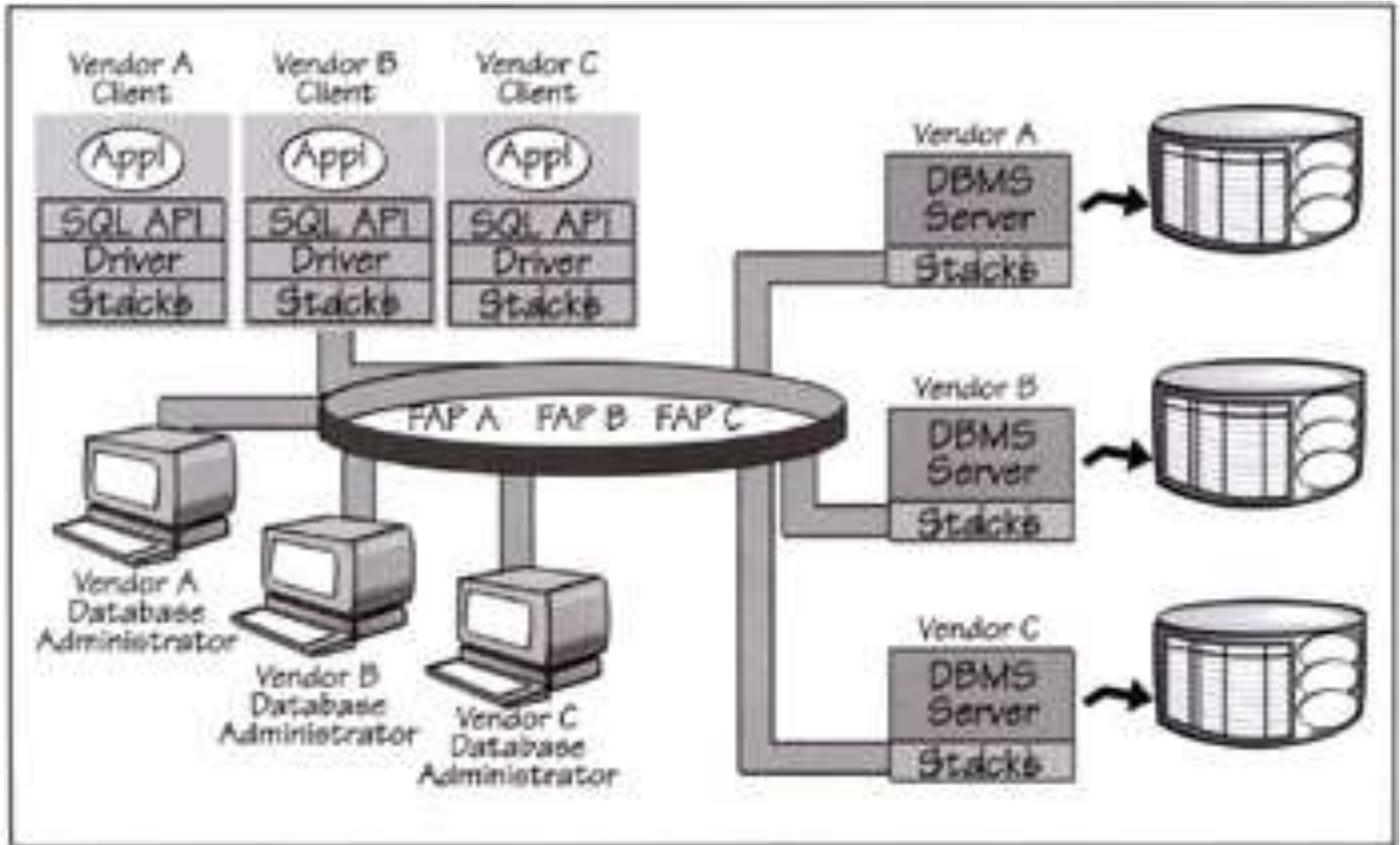
- Most vendors now support **multiple protocol stacks**.
- Some vendors bundle the stacks with their drivers; others support a common transport interface (like Sockets or Named Pipes) and require that you provide your own stacks
- At the server side, the vendor typically provides "listeners" for the different stacks.
- However, some vendors provide their own internal protocol gateways—for example, Oracle on Unix translate IPX/SPX packets to TCP/IP on the server side.

Gateways to other vendor databases

- Most vendor-supplied database gateways only provide an intersection of the features supported by the two databases—the least common denominator.
- This means that most vendor-supplied database gateways **only support dynamic SQL**.
- They are only good for simple data extracts and queries—not for transaction processing.

Client/Server database administration tools.

SQL integration/interoperability: multi vendor option



Challenges:

- Different SQL APIs
- Multiple database drivers
- Multiple FAPs and no solid interoperability
- Multiple administration tools
- Worse: federated database joins, federated commits, or concurrent access to federated data

- **Example:**

Customer data in **Database A**

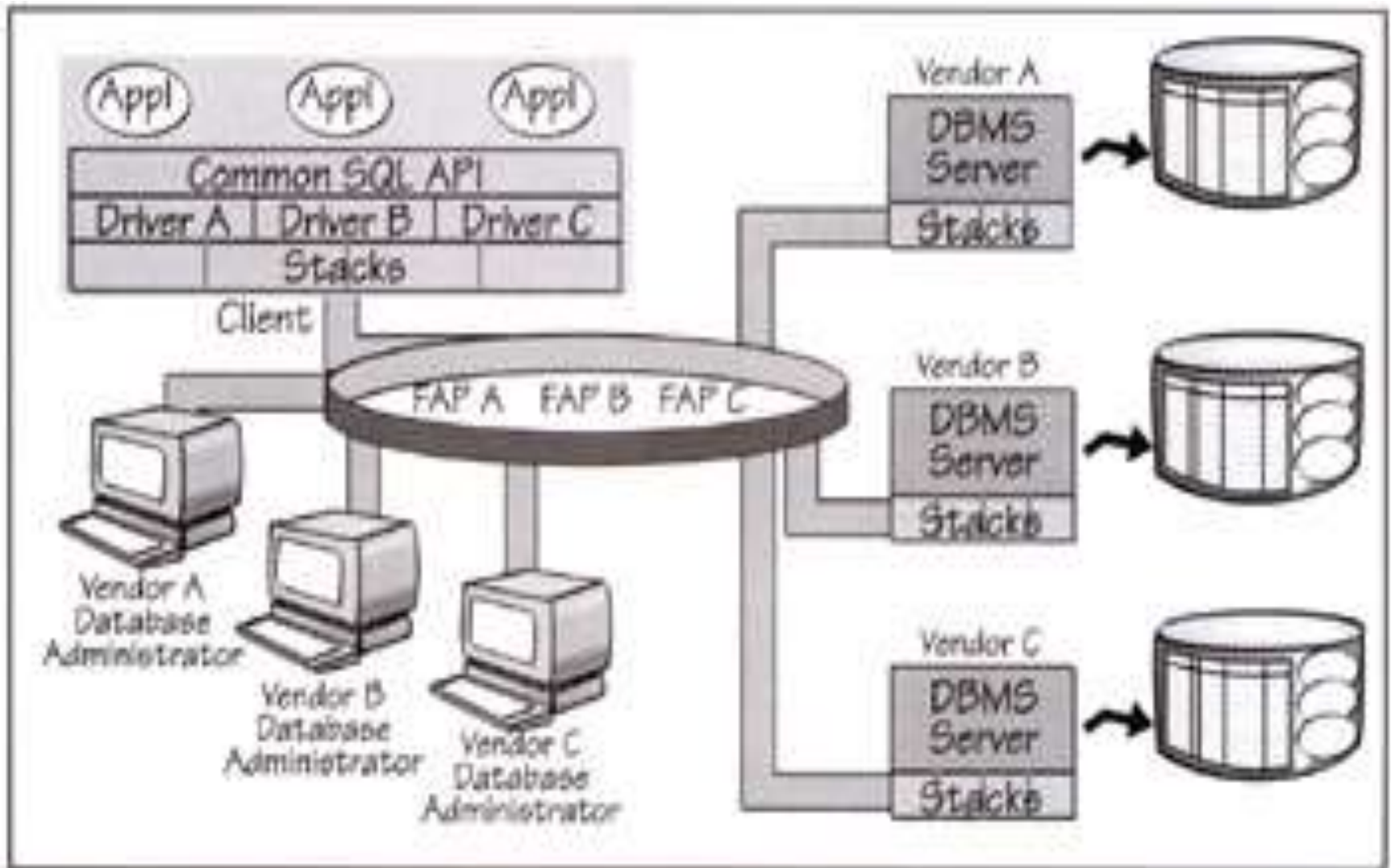
Order data in **Database B**

A query joins Customer and Order tables **across databases**

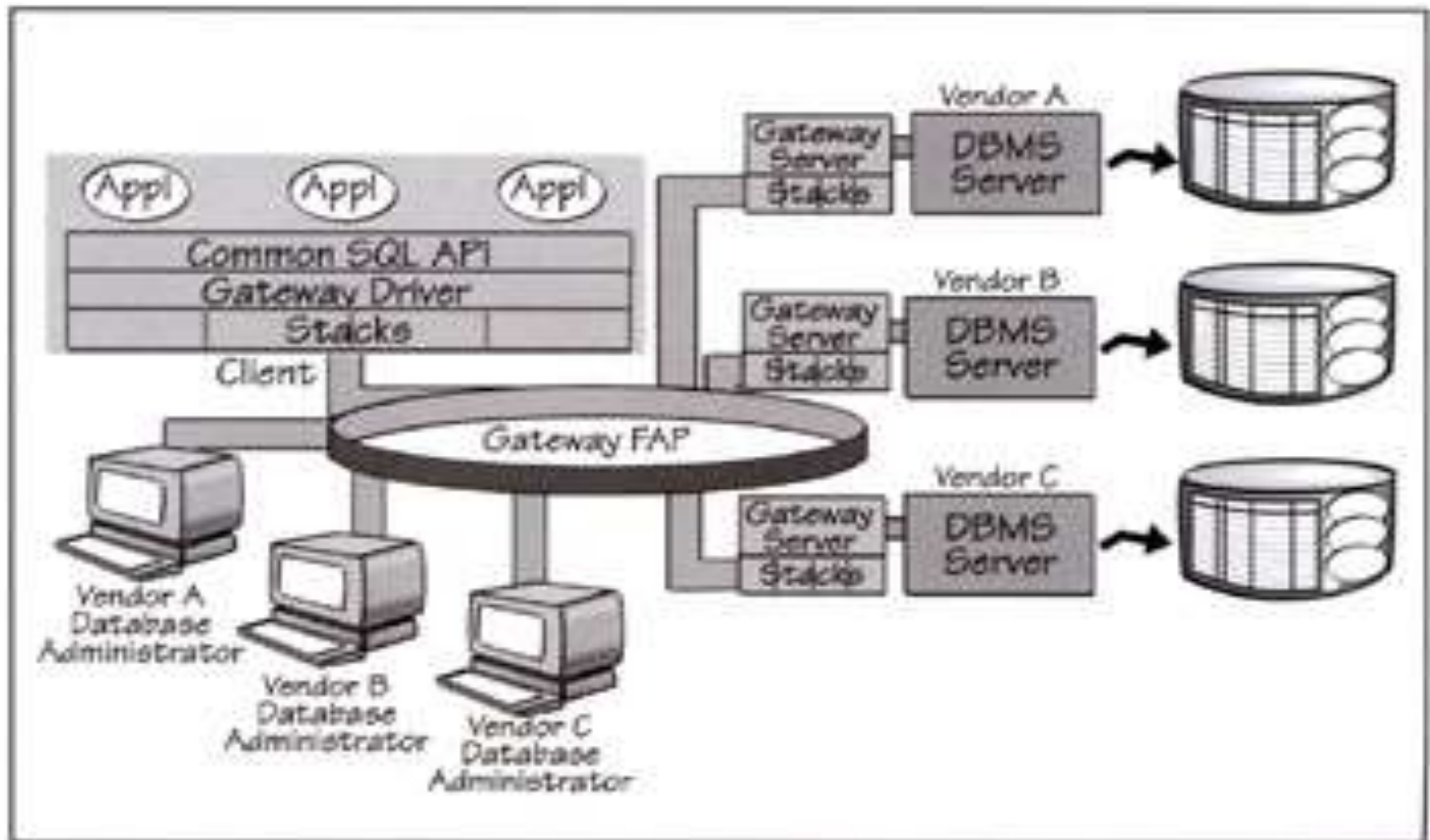
Key points:

- Databases remain **independent**
- Join is executed using **federation middleware**
- Increases transparency but may add **performance overhead**

Middleware Solution #1: The Common SQL Interface



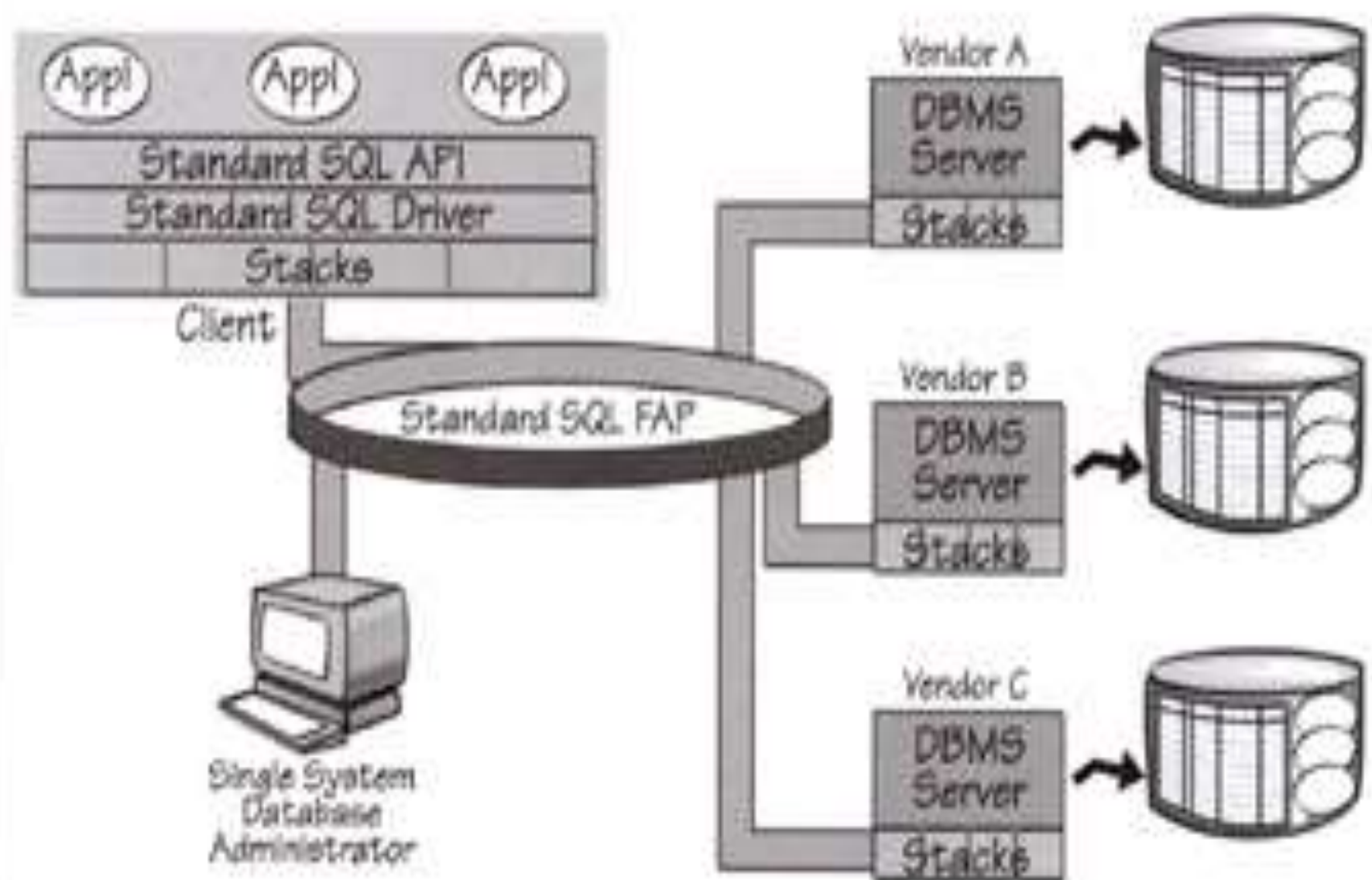
Middleware Solution #2: The Open SQL Gateway



Middleware Solution #3:

Federated

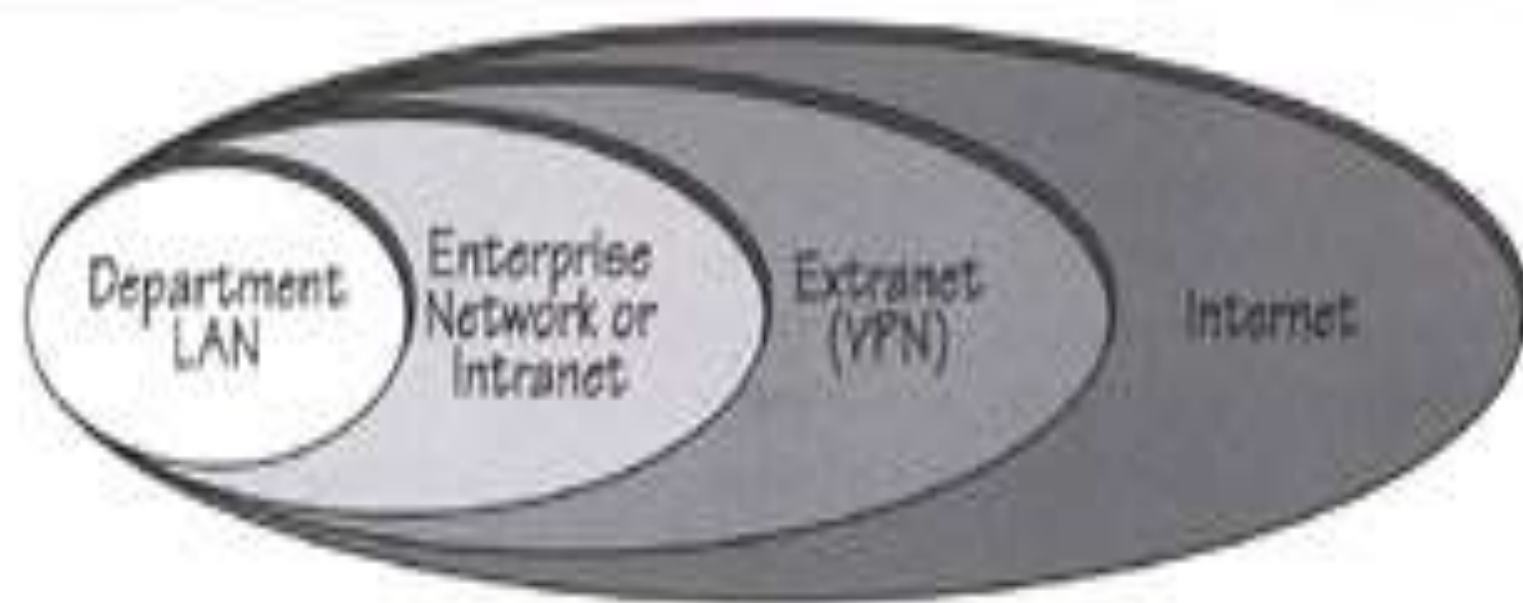
- To create a federated SQL environment that provides the same level of completeness as the single vendor implementation
- Remove the gateway catchers, which improves server performance, reduces cost, and simplifies maintenance.
- Create a single database administration interface



To eliminate the gateway catchers, **the common FAP must either support a superset of all the SQL dialects or it must tolerate native SQL dialects** (meaning that it must allow pass-throughs). The SQL vendors must also agree to replace their own private FAPs with the common FAP

Network Operating System (NOS) middleware

- **Goal: to provide a single system image across large networks**
- NOSs are evolving from being a collection of independent workstations, able to communicate via a shared file system, to becoming **real distributed computing environments that make the network transparent to users**



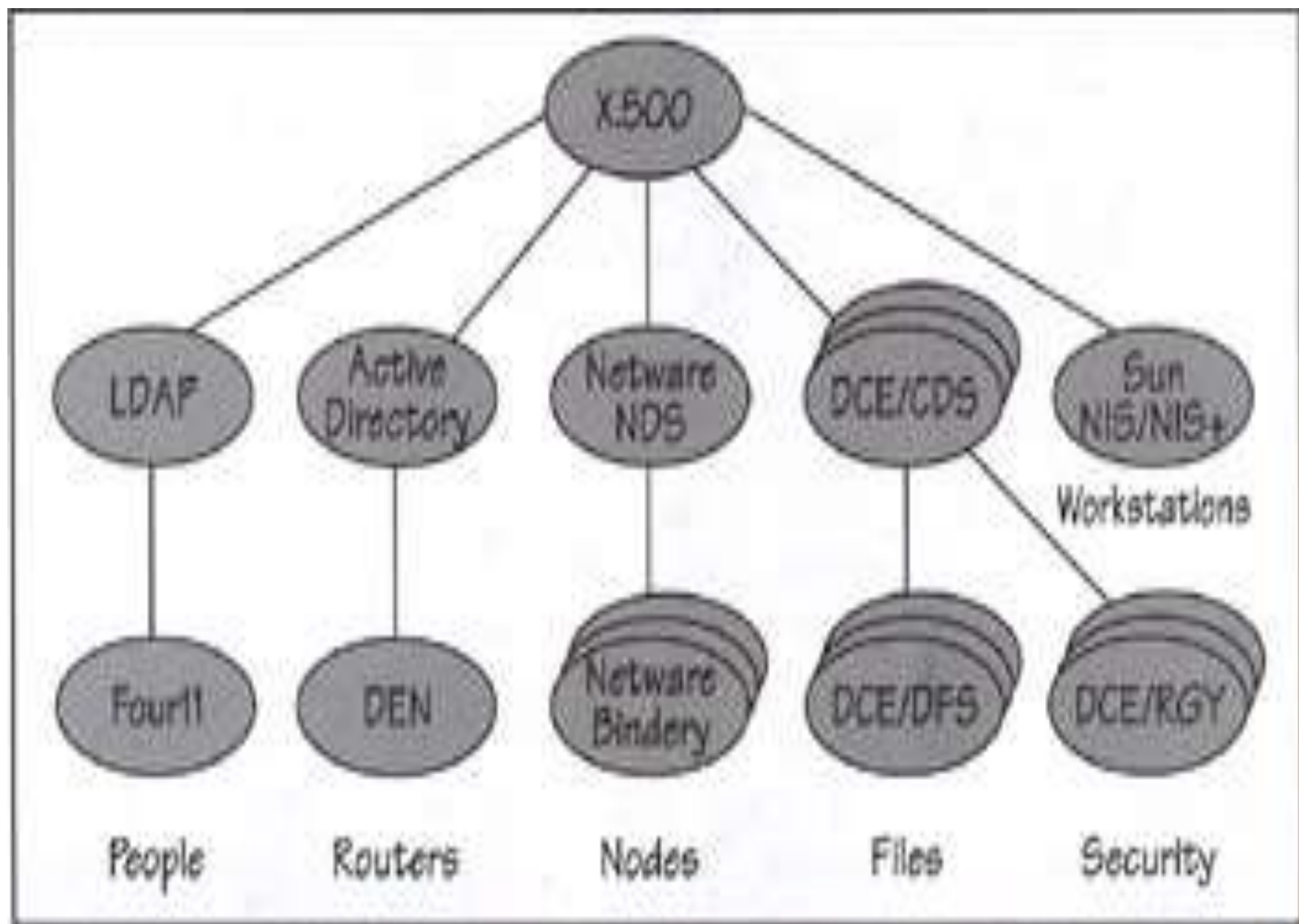
Transparencies desired:

Location transparency—

- You should not have to be aware of the location of a resource.
- Users should not have to include the location information in the resource's name.
- For example, `\\Machine\directory\file` surfaces the name of the server machine. This is a transparency violation.

Namespace transparency—

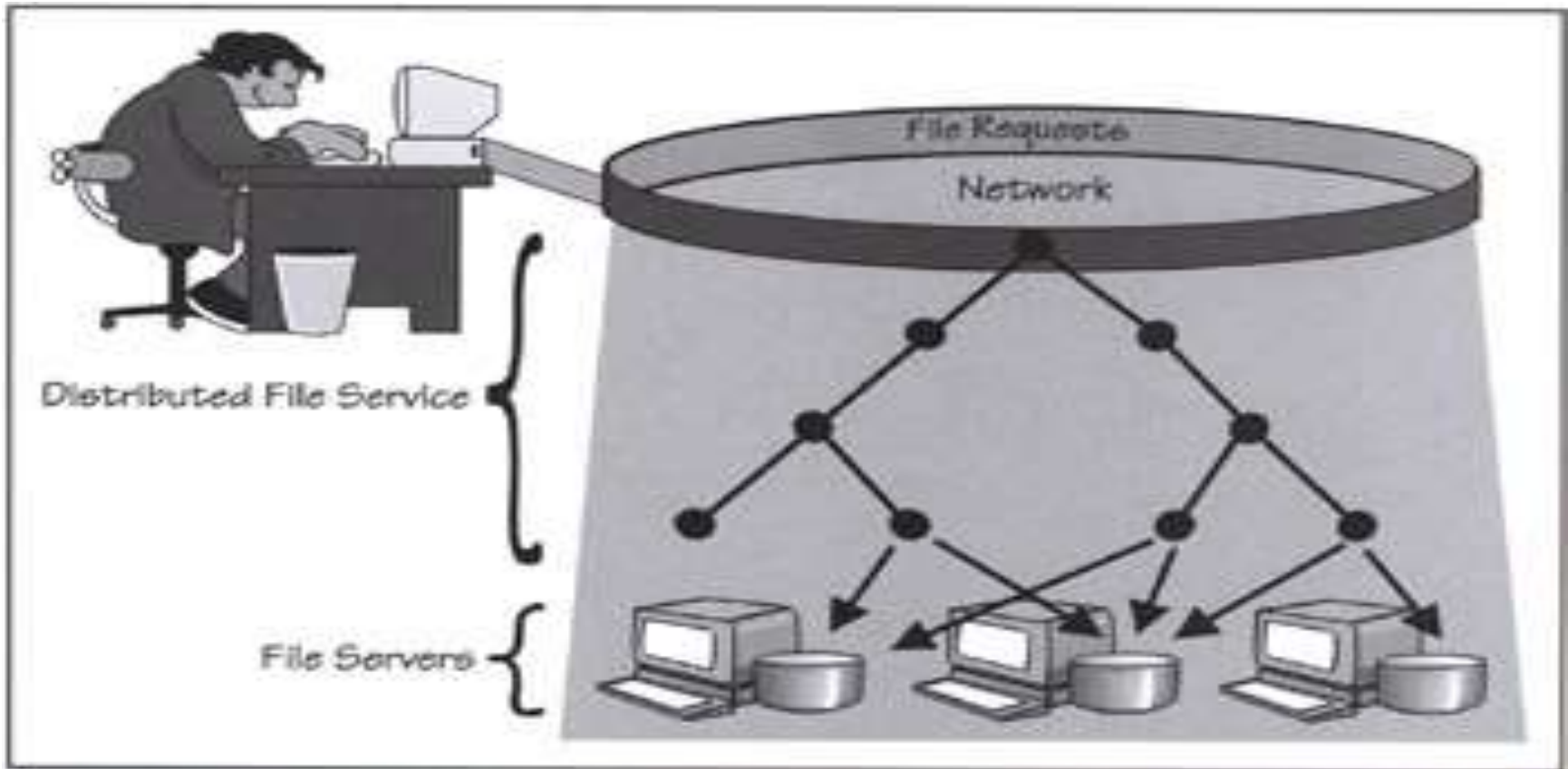
- Able to use the same naming conventions (and namespace) to locate any resource on the network. Universal namespace is shown below



- **Logon transparency**—You should be able to provide a single password (or authentication) that works on all servers and for all services on the network
- **Replication transparency**—You should not be able to tell how many copies of a resource exist.
- **Local/remote access transparency**—You should be able to work with any resource on the network as if it were on the local machine.
- **Distributed time transparency**—You should not see any time differences across servers. The NOS must synchronize the clocks on all servers.
- **Failure transparency**—You must be shielded from network failures. The NOS must handle retries and session reconnects. It must also provide some levels of service redundancy for fault-tolerance.
- **Administration transparency**—You should only have to deal with a single-system management interface. The NOS must be integrated with the local management services.

NOS: Extending the local OS' reach

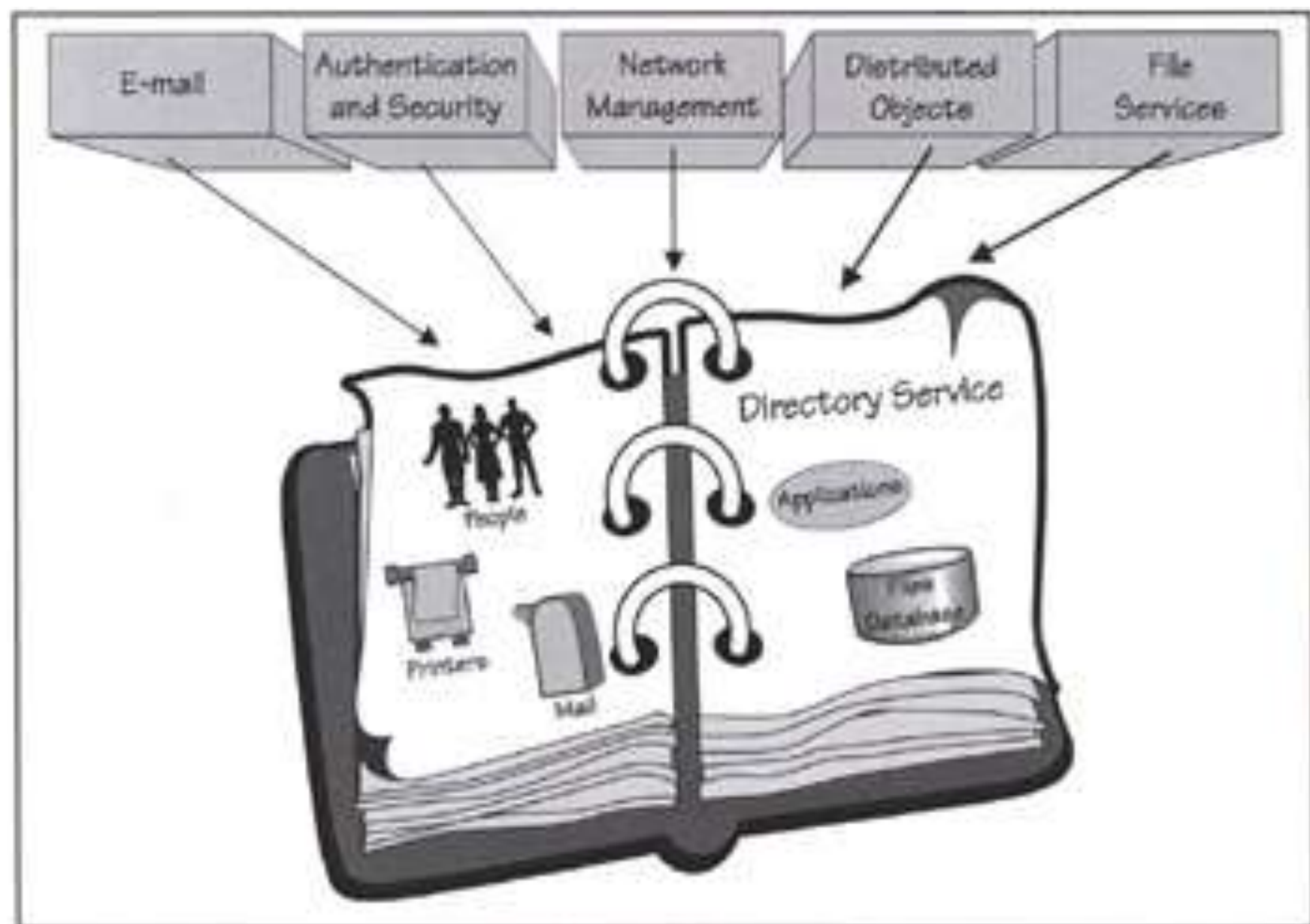
- **Goal:** Practically anything that can be done on a local OS can be done remotely and transparently.
- The NOS allows applications written for the local OS to become networked without changing a line of code
- Most NOSs allow clients that run on different OSs (such as DOS, Windows 98, Mac, and Unix) to share files and other devices
- the DCE Distributed File Service (DFS) provides a single-image file system that can be distributed across a group of file servers
- The DFS file naming scheme is **location-independent**.
- Each file has a unique identifier that is consistent across the network
- Files use the **DCE global namespace** just like the rest of the network resources. And the file system is integrated with the DCE security mechanisms.



- DFS provides a **Local File System (LFS)** with many advanced features, including replication facilities that make the file system highly available

Global Directory Services

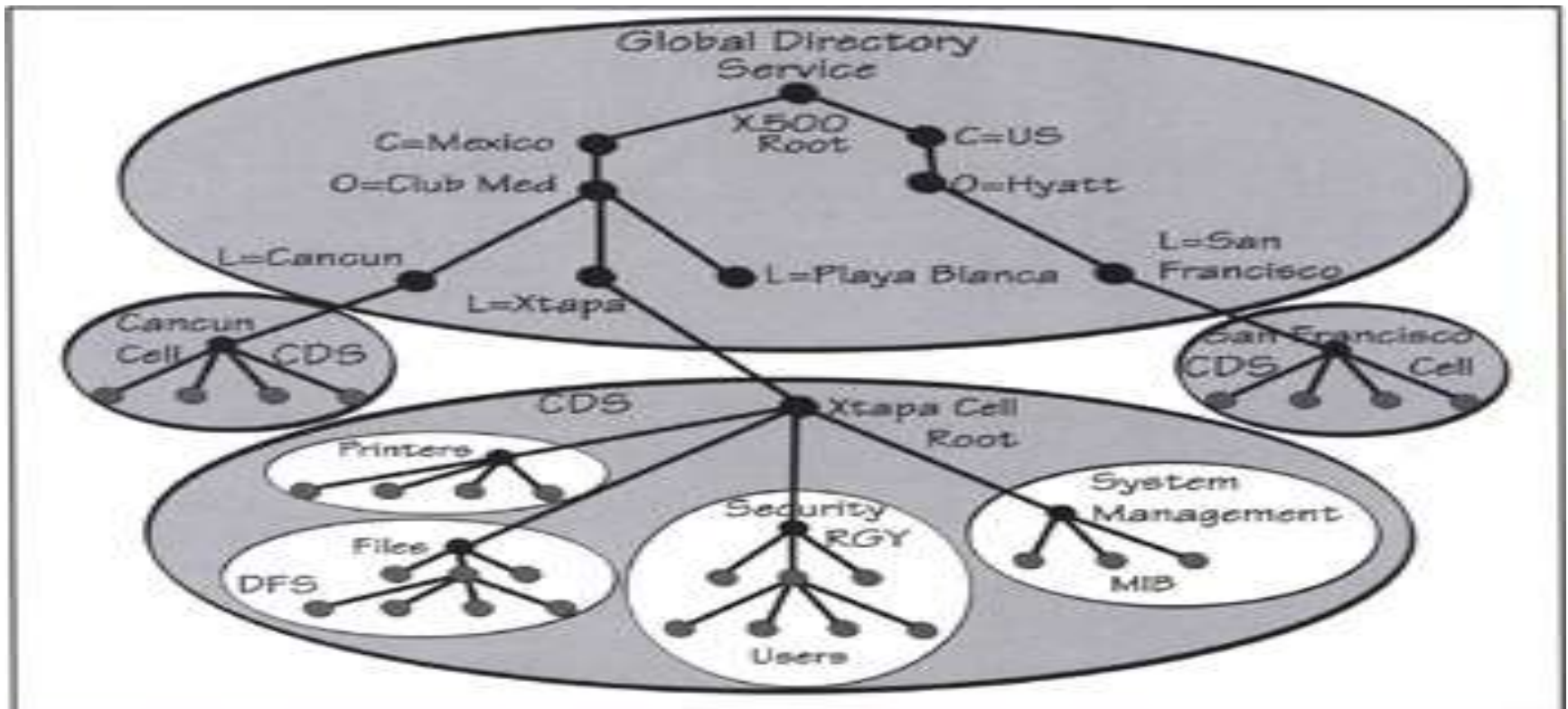
- The state of a client/server system is always in flux.
- Users join and leave the network.
- Services can be added and moved around at will.
Data is always being created and moved around
- Ideally, a distributed directory should provide a single image that can be used by all network applications—including **e-mail, system management, network inventory, file services, RPCs, distributed objects, databases, authentication, and security.**



- **A directory** lets you search its objects by **attribute**.
- **A schema describes the types of objects** a directory may contain; it also describes the attributes—both mandatory and optional—associated with an object type.
- Modern NOS directories have APIs and user interfaces that allow programs (or humans) to locate entities on the network by querying on the name or attributes.

Federated Directories—Creating a Unified Namespace.

Federated directories refer to a system where multiple independent directory services are linked together so they appear as one unified directory, while each directory remains autonomous and locally managed.



- Typically, a directory maintains a master copy and read-only shadow replicas.
- Two types of synchronization schemes are used to refresh the replicas:
- **Direct replication** causes any update to the master to be immediately shadowed on all replicas.
- **Skulking** causes a periodic propagation (for example, once a day) to all the replicas of all changes made on the master.
- NetWare Directory Service (NDS)—now supports multi-master updates.
- In contrast, LDAP (**Lightweight Directory Access Protocol**) is currently a single-master technology.

How Do You Interface to These Directories?

- **Directory-specific APIs and class libraries**
 - These provide interfaces that are tightly-coupled to a product such as Novell and Banyan provide APIs to their respective directories.
- **LDAP and X.500 APIs**
 - These are the standards-based interfaces defined by the IETF, X/Open, and the CCITT for the LDAP and X.500 directories
- **Java classes**
 - The JavaSoft-defined Java Naming and Directory Interface (JNDI) provides a set of generic Java classes that let you interface to a variety of directory services—such as LDAP, NDS, CORBA Naming, and NIS

- **Distributed object interfaces.**
 - ORB-based interfaces that let you talk to directory servers across ORBs
 - Typically, these interfaces let you pick your language, directory service, and platform
 - Example CORBA naming service (directory-neutral, language-neutral, and platform-neutral)
 - In the COM parallel universe, Microsoft defines the Active Directory Service Interface (ADSI). It is a directory-neutral and language-neutral interface for COM+ objects
- **Meta-directory services and scripts**
 - Meta-directories are glue products—from companies like Zoomit, NetVision, and WorldTalk—that synchronize data in existing directories via scripts.

X.500 and LDAP

- In the old days, a directory service was tied to a single application such as e-mail or to a specific operating system, such as NT domains.
- In contrast, a universal directory must be totally independent of both applications and NOSs.
- The original industry standard for universal directories was CCITT's X.500.
- However, X.500 proved to be too bulky and overly complicated.
- Today, the industry's new Esperanto for directories is LDAP—a downsized version of the original X.500.



THANK YOU